

Newsletter : ce qu'il reste à faire pour une version professionnelle

Branche analysée : `feature/request` (après fusion de `origin/staging`, commit `125e95d`) · Date : 8 octobre 2026
Commits concernés : `7a699fd`, `e65db44`, `3237433`, `c86e6f4` (EJM0101, 7 octobre 2026)

1. Synthèse

La newsletter possède une base solide : double opt-in, jetons hachés, tables protégées par RLS, campagnes bilingues, envoi par lots via Resend et journal d'envoi par destinataire. Le circuit complet **fonctionne en staging** : 2 abonnés actifs, 3 campagnes envoyées, aucun échec.

Il s'agit toutefois d'un **MVP**. Avant une mise en production, il faut corriger **5 problèmes bloquants**. Le plus grave : les liens de désinscription des anciens e-mails cessent de fonctionner. Il faut aussi traiter la conformité (preuve du consentement, désinscription en un clic exigée par Gmail et Yahoo, gestion des rebonds) et ajouter les fonctions attendues d'un outil d'e-mailing professionnel : éditeur riche, aperçu et envoi de test, statistiques, gestion des abonnés et programmation.

5

P0 : bloquant avant la production

7

P1 : conformité et délivrabilité

11

P2 : fonctions d'un outil pro

4

P3 : qualité technique

2. Périmètre et état actuel

Élément	Rôle
<code>supabase/migrations/00053_newsletter_and_email_campaigns.sql</code>	Tables <code>newsletter_subscribers</code> , <code>newsletter_campaigns</code> et <code>newsletter_campaign_deliveries</code> , avec RLS réservé au super-admin
<code>src/lib/newsletter/actions.ts</code>	Server action d'inscription publique (double opt-in, jeton valable 24 h)
<code>src/lib/newsletter/campaign-actions.ts</code>	Liste, création et envoi des campagnes (super-admin uniquement)
<code>src/app/api/newsletter/{confirm,unsubscribe}/route.ts</code>	Confirmation et désinscription (GET affiche un formulaire, POST applique l'action)
<code>src/lib/newsletter/http-response.ts</code>	Pages HTML brutes de confirmation et de désinscription
<code>src/lib/email/resend.ts</code>	Gabarit d'e-mail, envoi unitaire et envoi par lots (API Resend)

`src/app/[locale]/console/newsletter/`

Console : liste des campagnes, création d'un brouillon, envoi

`src/components/events/market/event-rail.tsx`

Seul formulaire d'inscription en ligne, sur la page Événements

Points déjà bien faits

- Double opt-in avec jetons aléatoires de 256 bits, stockés uniquement sous forme de hash SHA-256.
- GET → formulaire → POST : les scanners de liens des messageries ne confirment ni ne désinscrivent par erreur.
- RLS activé, aucune policy publique, écritures côté serveur uniquement, actions de console protégées par `requireSuperAdmin`.
- Passage atomique `draft/failed` → `sending` : une campagne ne part pas deux fois en cas de double clic.
- Journal par destinataire, et nouvelle tentative limitée aux destinataires en échec.
- Contenu bilingue envoyé selon la langue de l'abonné, version texte brut incluse, en-tête `List-Unsubscribe`.
- Validation zod des entrées et échappement HTML du contenu.

3. P0 : bloquant avant la production

P0-1 Les liens de désinscription des anciens e-mails ne fonctionnent plus

`campaign-actions.ts:164`, `actions.ts:63`

Chaque envoi de campagne génère un nouveau jeton et **écrase** `unsubscribe_token_hash`. Le lien « Se désabonner » de l'e-mail de confirmation et de toutes les campagnes précédentes affiche donc « Ce lien n'est plus valide ». Résultat : abonnés frustrés, signalements comme spam et non-conformité (RGPD et CAN-SPAM exigent une désinscription qui fonctionne).

À faire : utiliser un jeton stable par abonné, créé une fois pour toutes, ou mieux un jeton signé HMAC (`subscriber_id` + signature) qui ne demande aucun stockage et n'expire pas. Il faut aussi supprimer la mise à jour de jeton faite pour chaque abonné dans la boucle d'envoi.

P0-2 Désinscription en un clic (RFC 8058) non prise en charge

`resend.ts:199`, `unsubscribe/route.ts:19`

Depuis février 2024, Gmail et Yahoo exigent des expéditeurs en masse les en-têtes `List-Unsubscribe` et `List-Unsubscribe-Post: List-Unsubscribe=One-Click`. Seul le premier est envoyé. De plus, le POST de la route ne lit le jeton que dans le corps du formulaire, alors qu'un POST en un clic transmet le jeton dans l'URL : la requête serait rejetée.

À faire : ajouter l'en-tête `List-Unsubscribe-Post`, faire lire au POST le jeton depuis la query string en solution de repli, et ajouter un `mailto:` dans `List-Unsubscribe`.

P0-3 L'envoi synchrone ne passe pas à l'échelle et peut rester bloqué

`campaign-actions.ts:19` (`RECIPIENT_LIMIT = 500`), fonction `sendNewsletterCampaign`

Toute la campagne part dans une seule server action : une requête SQL par abonné, puis les appels Resend. Au-delà de 500 abonnés, l'envoi est **refusé**. Si la fonction serverless dépasse sa durée maximale, la campagne reste en statut `sending`. La console n'affiche alors aucun bouton pour la relancer.

À faire : passer à un envoi en arrière-plan avec file de lots (Supabase Queues/pgmq + cron, Inngest, QStash ou Vercel Cron). Le clic sur « Envoyer » met la campagne en file, un worker traite les lots de 100 et la console suit la progression. Prévoir aussi la récupération des campagnes bloquées en `sending` depuis plus de N minutes, puis supprimer la limite de 500.

P0-4 Configuration e-mail incohérente : risque de liens vers localhost

`resend.ts:5, 79, 180`, `actions.ts:25`, `campaign-actions.ts:20`, `src/lib/email/send.ts`, `.env.example`

- La newsletter lit `NEWSLETTER_FROM_EMAIL`, mais `.env.example` documente `EMAIL_FROM`, utilisé par `send.ts`. La variable de la newsletter n'est documentée nulle part.
- Les valeurs de repli de l'origine du site diffèrent : `http://localhost:3000` pour les liens de confirmation et de désinscription, `https://tradeindrc.net` pour le logo. Si `NEXT_PUBLIC_SITE_URL` manque en production, les e-mails pointent vers localhost.
- Deux wrappers Resend coexistent (`send.ts` et `resend.ts`).

À faire : centraliser l'origine du site dans un seul helper, sans repli vers localhost en production (erreur explicite à la place), documenter toutes les variables dans `.env.example` et fusionner les deux wrappers, idéalement avec le SDK officiel `resend`.

P0-5 Numéro de migration 00053 en double

`00053_fix_storage_owner_folder_policies.sql` et `00053_newsletter_and_email_campaigns.sql`

`supabase migration list` montre que la migration newsletter **n'est pas enregistrée** dans l'historique distant. Les tables existent pourtant en staging : elles ont donc été créées hors du flux normal. Le prochain `supabase db push` peut échouer, et les `CREATE TABLE` sans `IF NOT EXISTS` planteront sur une base où elles existent. Sur le futur projet de production (`brand-in-drc`), il faut vérifier si les tables ont bien été créées.

À faire : renuméroter en `00068_newsletter_and_email_campaigns.sql`, rendre la migration idempotente, puis exécuter `supabase migration repair` sur staging et l'appliquer sur la base de production.

4. P1 : conformité et délivrabilité

P1-1 La preuve du consentement n'est pas conservée

Le RGPD (art. 7) impose de pouvoir **démontrer** le consentement. Rien n'est enregistré : ni date, ni texte accepté, ni page d'origine, ni IP ou user-agent. Sur le composant `NewsletterSignup`, le mot « Politique de confidentialité » n'est même pas un lien.

À faire : ajouter les colonnes `consent_at`, `consent_text_version`, `source` et `consent_ip_hash`, faire un vrai lien vers `/privacy` et mentionner la newsletter (finalité, durée de conservation, droit d'opposition) dans la politique de confidentialité.

P1-2 Aucune gestion des rebonds ni des plaintes

Aucun webhook Resend n'est traité. On continue donc d'écrire aux adresses invalides et aux personnes qui ont signalé un spam, ce qui dégrade vite la réputation du domaine et peut entraîner la suspension du compte Resend.

À faire : créer une route `/api/webhooks/resend` avec vérification de signature (svix), qui traite `email.bounced`, `email.complained`, `email.delivered`, `email.opened` et `email.clicked`, et ajouter les statuts `bounced` et `complained` aux abonnés (liste de suppression).

P1-3 Inscription exposée aux robots et aux abus

`actions.ts:46`

Il n'y a ni CAPTCHA ni limitation par IP : un robot peut déclencher des e-mails de confirmation vers des milliers d'adresses, ce qui consomme le quota et dégrade la réputation. Le message « Cette adresse est déjà inscrite » permet en plus de **savoir si une personne est abonnée** (fuite d'information).

À faire : réutiliser le CAPTCHA déjà configuré (`NEXT_PUBLIC_CAPTCHA_SITE_KEY`), ajouter un champ honeypot et une limite de débit par IP, et toujours répondre « Vérifiez votre boîte mail ».

P1-4 Authentification du domaine expéditeur

Il faut vérifier et documenter SPF, DKIM (Resend) et DMARC (au moins `p=quarantine`, avec rapports). Il est aussi recommandé d'utiliser un sous-domaine dédié au marketing (par ex. `news.tradeindrc.com`), séparé des e-mails transactionnels, avec un nom d'expéditeur lisible et un `reply-to` surveillé.

P1-5 Mentions obligatoires manquantes dans les e-mails

Le pied de page ne contient pas d'adresse postale de l'expéditeur, obligatoire pour CAN-SPAM et attendue en Europe. Il manque aussi un lien vers la politique de confidentialité, un lien « gérer mes préférences » et la version « voir dans le navigateur ».

P1-6 Risque de doublons lors d'une nouvelle tentative

`campaign-actions.ts:197`

Si Resend accepte le lot mais que l'enregistrement du statut `sent` échoue, le lot est compté en échec et la nouvelle tentative le **renvoie** : les destinataires reçoivent l'e-mail deux fois.

À faire : envoyer l'en-tête `Idempotency-Key` de Resend (par ex. `campaignId:batchIndex`) et distinguer « envoyé mais non journalisé » de « rejeté par le fournisseur ».

P1-7 Durée de conservation et droit à l'effacement

Les inscriptions jamais confirmées (`pending`) ne sont jamais purgées, et le personnel n'a aucun outil pour supprimer un abonné à sa demande.

À faire : mettre en place un cron de purge des inscriptions non confirmées depuis plus de 30 jours, une action « supprimer définitivement » dans la console et une durée de conservation documentée.

5. P2 : fonctionnalités d'un outil professionnel

#	Manque	Ce qu'il faut ajouter
1	Éditeur de contenu	Le contenu est du texte brut échappé : pas de liens cliquables, d'images, d'intertitres ni de boutons. Il faut un éditeur riche (TipTap ou Markdown nettoyé), des gabarits réutilisables (à la une, opportunités, événements) et un rendu fiable avec <code>react-email</code> .
2	Aperçu et e-mail de test	Aperçu FR/EN fidèle (bureau et mobile), bouton « M'envoyer un test », et nombre exact de destinataires dans la confirmation d'envoi, au lieu de <code>window.confirm</code> .
3	Cycle de vie des brouillons	Modifier, dupliquer, supprimer et archiver une campagne : seule la création existe aujourd'hui.
4	Programmation	Champ <code>scheduled_at</code> , envoi déclenché par cron, possibilité d'annuler une campagne programmée.
5	Gestion des abonnés	Page console avec compteurs (actifs, en attente, désinscrits, rebonds), recherche, export CSV, import avec preuve de consentement, désinscription et suppression manuelles.
6	Statistiques de campagne	Délivrés, ouvertures, clics, rebonds, désinscriptions et plaintes, par campagne et dans le temps (via les webhooks P1-2), ainsi qu'un graphique de croissance des abonnés.
7	Segmentation	Thèmes au choix (opportunités, événements, secteurs, actualités), profil acheteur ou exportateur, pays. Ciblage d'une campagne par segment.
8	Centre de préférences	Page où l'abonné change sa langue et ses thèmes ou suspend ses envois, au lieu d'une simple désinscription totale.
9	Pages et e-mail de bienvenue	Les pages de confirmation sont du HTML brut hors de l'application, avec un lien « Retour aux événements » (<code>http-response.ts:33</code>). Il faut de vraies pages <code>/[locale]/newsletter/...</code> au design du site, ainsi qu'un e-mail de bienvenue après confirmation.
10	Points d'inscription	L'inscription n'existe que sur la page Événements. Il faut l'ajouter au footer, à la page d'accueil, aux actualités et au blog, sur une page dédiée <code>/newsletter</code> , et proposer une case opt-in à la création de compte. Le composant <code>NewsletterSignup</code> est exporté mais inutilisé.
11	Offre « Visibilité newsletter »	<code>promotion-plans.ts:84</code> vend une visibilité dans la newsletter, mais aucun mécanisme ne permet de mettre une entreprise premium en avant dans une campagne. Il faut un bloc « Entreprise à la une » sélectionnable dans l'éditeur.

Gouvernance (recommandé) : un modérateur rédige et un super-admin valide puis envoie, avec un journal d'audit (qui a créé, modifié, envoyé).

6. P3 : qualité technique

- **Tests** : aucun pour l'instant. Prévoir des tests unitaires (jetons, validation), des tests d'intégration de l'envoi avec Resend simulé et un test end-to-end inscription → confirmation → désinscription.

- **Observabilité :** remplacer les `console.error` par un suivi d'erreurs (Sentry) et alerter le personnel quand une campagne échoue.
- **Rendu des e-mails :** tester sur Outlook, Gmail, Apple Mail et en mode sombre (Litmus ou Email on Acid), et ajouter un preheader distinct du sujet.
- **Langues :** les contraintes `CHECK (locale IN ('en', 'fr'))` et les colonnes `_en/_fr` devront évoluer quand TR, ZH et ES seront activées.

7. Feuille de route proposée

Phase	Contenu	Effort indicatif
1. Mise en production sûre	P0-1 à P0-5, P1-3 (CAPTCHA, anti-énumération) et P1-6 (idempotence)	3 à 4 jours
2. Conformité et délivrabilité	Preuve du consentement et politique de confidentialité, webhooks Resend avec liste de suppression, DNS SPF/DKIM/DMARC, mentions légales, purge et effacement	3 à 5 jours
3. Outil professionnel	Éditeur riche et gabarits, aperçu et test, cycle de vie des brouillons, programmation, gestion des abonnés, statistiques, pages localisées et e-mail de bienvenue, nouveaux points d'inscription	2 à 3 semaines
4. Croissance	Segmentation et centre de préférences, entreprises premium à la une, workflow de validation, tests A/B des sujets, e-mails automatiques (nouvelles opportunités par secteur)	2 à 3 semaines

Les estimations supposent un développeur qui connaît déjà la base de code. Elles sont indicatives et incluent les tests de base.

8. Vérifications effectuées

- Lecture complète du code des commits `7a699fd`, `e65db44`, `3237433` et `c86e6f4`.
- `supabase migration list` sur le projet lié (staging) : 00053 newsletter absente de l'historique distant.
- Lecture seule des tables en staging (comptages par statut, sans données personnelles) : 2 abonnés actifs (1 EN, 1 FR), 3 campagnes `sent`, 4 envois réussis, 0 échec.
- Non vérifié : configuration DNS du domaine, variables d'environnement Vercel et présence des tables sur le projet de production `brand-in-drc`.